



Tecnológico de Monterrey

Campus Querétaro

Evidencia Demostración de un Paradigma de Programación

Inaki Mancera Llano - A01708827

Materia: Implementacion de Metodos Computacionales

Fecha de entrega
22 de Mayo 2026

1. Problema Seleccionado

Para esta evidencia se seleccionó el problema de Codeforces 1158A - The Party and Sweets. El problema plantea una situación donde hay n niños y m niñas en una fiesta. Cada niño le da cierta cantidad de dulces a cada niña, por lo que se puede imaginar una matriz de n filas y m columnas. Cada fila representa a un niño y cada columna representa a una niña. (Codeforces, 2019)

Por ejemplo, si hay 3 niños y 2 niñas, la matriz tendría esta forma:

	niña 1	niña 2
niño 1	?	?
niño 2	?	?
niño 3	?	?

Cada posición de la matriz representa cuántos dulces le dio un niño a una niña. Sin embargo, el problema no proporciona directamente todos los valores de la matriz. En lugar de eso, proporciona dos listas: una lista de niños y una lista de niñas.

La lista de niños representa el mínimo de cada fila. Es decir, si **boys** = [1, 2, 1], significa que el niño 1 debe tener como mínimo de su fila el valor 1, el niño 2 debe tener como mínimo el valor 2 y el niño 3 debe tener como mínimo el valor 1.

La lista de niñas representa el máximo de cada columna. Es decir, si **girls** = [3, 4], significa que la niña 1 debe tener como máximo de su columna el valor 3 y la niña 2 debe tener como máximo de su columna el valor 4.

Para este caso un ejemplo de matriz válida sería el siguiente:

	niña 1	niña 2
niño 1	1	4
niño 2	3	2
niño 3	1	1

Revisando las filas para confirmar que se cumplan estas condiciones tenemos que:

niño 1: $\min(1, 4) = 1$

niño 2: $\min(3, 2) = 2$

niño 3: $\min(1, 1) = 1$

Esto coincide con **boys** = [1, 2, 1].

Revisando las columnas para confirmar que se cumplan estas condiciones tenemos que:

niña 1: $\max(1, 3, 1) = 3$

niña 2: $\max(4, 2, 1) = 4$

Esto coincide con **girls = [3, 4]**.

La suma total de dulces en esta matriz es:

$$1 + 4 + 3 + 2 + 1 + 1 = 12$$

Por lo que la respuesta para este caso es **12**.

El objetivo del problema es encontrar la menor suma total posible de dulces en una matriz que cumpla las condiciones de mínimos por fila y máximos por columna. Si no existe ninguna matriz que cumpla esas condiciones, la respuesta debe ser **-1**.

Siendo que la condición principal para saber si el problema es posible es que:

- $\max(\text{boys}) \leq \min(\text{girls})$

Si el mayor mínimo de los niños es mayor que el menor máximo de las niñas, existe una contradicción. Por ejemplo, si un niño necesita que todos sus valores sean al menos 5, pero una niña no puede recibir más de 4, entonces no se puede construir una matriz válida.

2. Algoritmo para solución

La estrategia del algoritmo consiste en iniciar con la matriz más barata posible, usando los valores mínimos de los niños en cada fila. Después, se ajustan algunas posiciones para que cada columna alcance el máximo requerido por las niñas. Para minimizar el costo, se usa principalmente el niño con el mayor valor en boys, ya que partir de ese valor genera el menor aumento posible. Si **max(boys)** es menor que **min(girls)**, se necesita un ajuste adicional usando el segundo mayor valor de boys, para asegurar que el niño con mayor mínimo conserve al menos una posición con su valor mínimo original.

Siguiendo el ejemplo antes dado, la matriz más barata con solo valores de boys:

	niña 1	niña 2
niño 1	1	1
niño 2	2	2
niño 3	1	1

Lo que cuesta:

$$\text{dulces} = 2 * (1 + 2 + 1) = 8$$

La fórmula siendo niñas o columnas por la suma de dulces de niños **boys = [1, 2, 1]**:

$$m * \text{sum}(\text{boys})$$

Pero las columnas todavía no tienen cada una los máximos requeridos, entonces se usa al niño con el mayor mínimo para que aumente ;lo menos posible la cantidad de dulces y si el máximo de niños no es igual si no menor que el mínimo de niñas, se tiene que usar el segundo máximo de niños para uno de los valores de niñas

Se calcula lo que se le suma al total de dulces ahora con los de niñas **girls = [3, 4]**:

$$(3 - 2) + (4 - 2) = 3$$

$$dulces = 8 + 3 = 11$$

Y como el **max(boys)** es menor que **min(girls)**, al total de dulces se le agrega la diferencia entre **max(boys)** y **secondMax(boys)**:

$$2 - 1 = 1$$

Y se le suma a el total de dulces para tener la cantidad final de dulces requeridos como se puede observar en la matriz válida:

$$11 + 1 = 12$$

	niña 1	niña 2
niño 1	1	4
niño 2	3	2
niño 3	1	1

Teniendo todo esto en cuenta, la fórmula final sería la siguiente:

n = cantidad de niños

m = cantidad de niñas

$sumB$ = suma de boys

$sumG$ = suma de girls

$maxB$ = máximo de boys

$minG$ = mínimo de girls

$secondMaxB$ = segundo mayor valor de boys

Si **$maxB > maxG$** regresa un -1, y si esto no se cumple se sigue al valor base de dulces mínimos:

$$base = (m * sumB) + (sumG - m * maxB)$$

Y si **$maxB < minG$** la respuesta sería la siguiente (que se queda solo como base si son iguales):

$$respuesta = base + (maxB - secondMaxB)$$

3. Paradigma funcional

La solución se implementa bajo el paradigma funcional porque el problema se resuelve mediante funciones que reciben datos y devuelven resultados sin modificar variables globales. Las listas de niños y niñas se procesan mediante funciones auxiliares recursivas como **sum-list** y **merge-sort**. En lugar de usar ciclos o variables acumuladoras mutables, la solución divide el problema en funciones pequeñas: una para sumar listas, otra para dividir listas, otra para combinar listas ordenadas y otra para aplicar la fórmula principal del problema. Esto permite expresar el algoritmo como una transformación de datos. (Racket Documentation, n.d.)

4. Implementación

La implementación se realizó en DrRacket. El archivo principal **PyS.rkt** contiene la función **party-and-sweets**, que recibe directamente las listas **boys** y **girls**. También contiene funciones auxiliares como **sum-list** para sumar listas recursivamente, **split-list** para dividir una lista en dos partes, **merge** para combinar dos listas ordenadas y **merge-sort** para ordenar las listas. (GeeksforGeeks, 2025)

El archivo **pruebas.rkt** contiene pruebas automatizadas. Estas pruebas llaman directamente a **party-and-sweets** con listas ya definidas, sin depender de entrada manual desde terminal.

5. Pruebas

Caso	boys	girls	Resultado esperado	Razón
Caso explicado	[1, 2, 1]	[3, 4]	12	Matriz de dulces mínima válida
Igualdad (maxB, minG)	[2, 3]	[3, 3]	10	maxB = minG
Imposible	[3, 5]	[4, 6]	-1	maxB > minG
Un niño imposible	[2]	[3, 4]	-1	No se conserva un valor de maxB en su fila
Un niño valido	[3]	[3, 4]	7	Se conserva un valor de maxB en su fila

Cada prueba utiliza **equal?** para comparar el resultado obtenido por **party-and-sweets** contra el resultado esperado.

6. Complejidad temporal/espacial solución funcional en Racket

En la solución funcional la complejidad temporal presente es la siguiente:

sum – *list boys* → $O(n)$

sum – *list girls* → $O(m)$

merge – *sort boys* → $O(n \log n)$

merge – *sort girls* → $O(m \log m)$

La parte dominante siendo el ordenamiento, por lo que la complejidad temporal sería:

$O(n \log n + m \log m)$

Y como la complejidad espacial es lineal en ambos casos (recorrido de lista y en mergesort), la complejidad espacial sería:

$O(n + m)$

La solución alternativa con scripting tendría la misma complejidad temporal, porque usaría el mismo algoritmo general: ordenar las listas, calcular sumas y aplicar condiciones.

7. Solución alternativa (scripting en Python).

Como paradigma alternativo, el problema podría resolverse mediante scripting en Python. En este enfoque, la solución se organizaría como un script que lee la entrada, convierte los datos a listas, ordena las listas y calcula los valores necesarios usando funciones integradas como `sum()`, `max()`, `min()` y `sort()`.

A diferencia de la solución funcional en DrRacket, donde la lógica se expresa como composición de funciones recursivas sobre listas, en scripting la solución estaría enfocada en escribir una secuencia directa de pasos: leer datos, procesarlos y mostrar la respuesta. Este enfoque es común en scripts pequeños porque permite automatizar tareas de entrada, procesamiento y salida de manera breve.

La lógica algorítmica sería la misma: validar si `max(boys)` es mayor que `min(girls)`, calcular la fórmula base y aplicar el ajuste con el segundo máximo cuando sea necesario.

8. Comparación de paradigmas

Aspecto	Funcional en DrRacket	Scripting en Python
Organización	Funciones recursivas	Script secuencial

Suma de listas	sum-list recursivo	sum()
Ordenamiento	merge-sort	sort()
Estado mutable	No	Se puede usar
Complejidad temporal	$O(n \log n + m \log m)$	$O(n \log n + m \log m)$
Complejidad espacial	$O(n + m)$	$O(n + m)$

9. Referencias

- Codeforces. (2019). Problem 1158A - The Party and Sweets. <https://codeforces.com/problemset/problem/1158/A>
- Racket Documentation. (n.d.). Lists, iteration, and recursion. The Racket Guide. https://docs.racket-lang.org/guide/Lists__Iteration__and__Recursion.html
- GeeksforGeeks. (2025). Merge sort. <https://www.geeksforgeeks.org/dsa/merge-sort/>